

IF2224 Teori Bahasa Formal dan Otomata

Milestone 4



13524121 - Billy Ontoseno Irawan

13524122 - Nathaniel Christian

13524139 - Azri Arzaq Pohan

13524142 - Rasyad Satyatma

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2026

BAB 1

Dasar Teori

1.1 Intermediate Code Generation

Setelah *frontend* menyelesaikan proses analisis dan validasi program, tahap pertama pada *backend* adalah menerjemahkan representasi konseptual program tersebut ke dalam bentuk instruksi yang dapat dieksekusi oleh mesin. Proses translasi ini tidak dilakukan secara langsung dari kode sumber, melainkan berdasarkan informasi semantik yang telah dikumpulkan dan diverifikasi pada tahap sebelumnya. Oleh karena itu, ***Decorated Abstract Syntax Tree (Decorated AST)*** berperan sebagai sumber informasi utama yang menjadi dasar bagi seluruh proses pembangkitan kode. Seluruh informasi mengenai struktur sintaksis, tipe data, *scope*, serta atribut semantik lainnya yang telah dilekatkan pada setiap simpul AST digunakan untuk menghasilkan *intermediate representation* yang akurat dan konsisten dengan makna program asal.

1.1 *Decorated AST*

Decorated AST adalah AST yang telah diberikan anotasi semantik. Setiap node yang relevan menyimpan atribut tambahan, seperti tipe hasil ekspresi (integer, real, boolean, dan sebagainya), indeks entri di *symbol table*, informasi blok atau level leksikal, dan parameter lain yang diperlukan untuk tahap generasi kode. Dengan *Decorated AST*, generator dapat dengan cerdas memutuskan instruksi Intermediate Code spesifik yang harus dihasilkan.

1.2 *Three-Address Code*

TAC adalah representasi perantara yang digunakan dalam desain kompiler untuk menyederhanakan ekspresi kompleks dengan memecahnya menjadi serangkaian instruksi sederhana. Setiap instruksi berisi paling banyak tiga alamat: dua operan dan satu hasil.

Hasil dari setiap operasi biasanya disimpan dalam variabel sementara. TAC membuat urutan operasi menjadi eksplisit. Hal ini membantu dalam optimasi kode dan menyederhanakan pembuatan *machine code*.

1.3 Translasi *Control Flow*

Transformasi ekspresi aritmatika ke dalam bentuk *Three-Address Code* (TAC) umumnya relatif sederhana karena mengikuti pola sintaksis yang deterministik. Sebaliknya, tantangan utama dalam proses *Intermediate Code Generation* terletak pada representasi *control flow* seperti percabangan (if-else) dan perulangan (while) dalam bentuk instruksi yang bersifat linier.

Permasalahan yang muncul adalah bagaimana mengubah struktur program yang tersusun secara hierarkis dan bersarang menjadi urutan instruksi yang dapat dieksekusi secara berurutan.

Untuk mengatasi hal tersebut, TAC memanfaatkan dua mekanisme utama, yaitu label dan instruksi lompatan (jump).

Label merupakan penanda statis yang menunjukkan posisi tertentu dalam urutan instruksi TAC, misalnya L1: atau L_END:.

Conditional Jump digunakan untuk mengalihkan alur eksekusi ke suatu label apabila kondisi tertentu terpenuhi atau tidak terpenuhi. Dalam TAC, bentuk yang umum digunakan adalah IF_FALSE condition GOTO label.

Unconditional Jump atau GOTO digunakan untuk memindahkan alur eksekusi secara langsung ke suatu label tanpa mempertimbangkan kondisi apa pun.

Pada struktur percabangan, alur program harus memilih salah satu dari dua cabang eksekusi berdasarkan hasil evaluasi suatu kondisi. Proses translasi ke TAC dilakukan dengan mengevaluasi kondisi terlebih dahulu, kemudian menggunakan instruksi lompatan untuk menentukan cabang yang akan dieksekusi. Struktur perulangan memiliki kompleksitas tambahan karena setelah blok perulangan selesai dieksekusi, alur program harus kembali ke bagian awal untuk melakukan evaluasi ulang terhadap kondisi perulangan.

Melalui mekanisme label dan lompatan ini, struktur kendali yang semula berbentuk blok bersarang dapat direpresentasikan sebagai rangkaian instruksi linier. Dengan demikian, tahap eksekusi tidak lagi memerlukan pemahaman khusus mengenai konsep percabangan atau perulangan, melainkan cukup menjalankan instruksi secara berurutan serta mengikuti arahan lompatan yang ditentukan oleh TAC.

1.4 Arsitektur Mesin Eksekusi

Setelah *Intermediate Code Generator* berhasil mentransformasikan *Decorated AST* menjadi sekumpulan instruksi *Three-Address Code* (TAC), proses translasi program dapat dianggap selesai. Tahap berikutnya adalah eksekusi program yang dilakukan oleh komponen Interpreter. Dalam konteks ini, interpreter dapat dipandang sebagai sebuah *Virtual Machine* (VM) yang bertugas menjalankan instruksi TAC yang dihasilkan oleh tahap kompilasi sebelumnya.

Meskipun tidak memiliki bentuk fisik seperti prosesor pada komputer nyata, virtual machine tetap mengimplementasikan konsep-konsep fundamental yang sama, seperti memori, alur kontrol, serta siklus eksekusi instruksi. Sebagaimana sebuah Central Processing Unit (CPU), interpreter mengeksekusi instruksi secara berurutan. Untuk mengetahui instruksi yang sedang dieksekusi, interpreter menggunakan sebuah penunjuk khusus yang disebut *Instruction Pointer* (IP) atau *Program Counter* (PC). Nilai IP menunjukkan posisi instruksi TAC yang sedang diproses pada suatu waktu tertentu.

Proses eksekusi berlangsung melalui siklus yang terdiri atas tiga tahap utama:

1. Fetch
Interpreter mengambil instruksi TAC yang saat ini ditunjuk oleh Instruction Pointer.
2. Decode
Instruksi yang telah diambil dianalisis untuk menentukan jenis operasi yang harus dilakukan, operand yang terlibat, serta informasi lain yang diperlukan untuk eksekusi.

3. Execute

Interpreter melaksanakan operasi yang direpresentasikan oleh instruksi tersebut. Sebagai contoh, operasi aritmatika akan menghasilkan nilai baru, sedangkan assignment akan menyimpan nilai ke dalam memori.

Setelah tahap Execute selesai, nilai IP akan diperbarui agar menunjuk ke instruksi berikutnya. Pada kondisi normal, pembaruan dilakukan dengan menambahkan satu pada nilai IP. Namun, apabila instruksi yang dieksekusi merupakan instruksi lompatan seperti GOTO, maka nilai IP akan langsung diubah menuju alamat instruksi yang direferensikan oleh label tujuan. Siklus ini akan terus berulang hingga seluruh instruksi TAC selesai dieksekusi atau interpreter mencapai kondisi penghentian program.

Agar dapat menyimpan variabel, hasil perhitungan, dan informasi eksekusi lainnya, interpreter memerlukan suatu mekanisme manajemen memori. Salah satu struktur data yang paling penting dalam arsitektur interpreter adalah Stack. Stack merupakan struktur data yang bekerja berdasarkan prinsip Last In, First Out (LIFO), yaitu elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan. Operasi utama pada stack adalah Push - menambahkan elemen ke bagian atas stack - dan Pop - menghapus elemen dari bagian atas stack.

Penggunaan stack tidak hanya bertujuan untuk menyimpan data sementara, tetapi juga berperan penting dalam pengelolaan scope dan pemanggilan fungsi. Setiap kali program memasuki ruang lingkup baru atau memanggil sebuah fungsi, interpreter akan membuat sebuah unit memori yang disebut Stack Frame. Stack frame merepresentasikan konteks eksekusi yang sedang aktif pada saat itu. Secara umum, sebuah stack frame dapat berisi:

- Variabel lokal yang dimiliki fungsi
- Parameter atau argumen yang diteruskan saat pemanggilan fungsi
- *Return address*
- Data lain yang diperlukan selama eksekusi fungsi berlangsung

Ketika suatu fungsi dipanggil, stack frame baru akan ditambahkan ke puncak stack melalui operasi push. Selama fungsi tersebut aktif, interpreter hanya akan mengakses data yang berada pada frame paling atas. Setelah fungsi selesai dieksekusi, stack frame akan dihapus melalui operasi pop sehingga seluruh variabel lokal yang tersimpan di dalamnya secara otomatis tidak dapat diakses lagi.

Mekanisme ini memungkinkan setiap fungsi memiliki ruang lingkup (scope) yang terisolasi dari fungsi lain. Akibatnya, variabel lokal yang dideklarasikan dalam suatu fungsi tidak dapat diakses secara langsung oleh fungsi lain, sekaligus memungkinkan penggunaan memori yang lebih efisien.

Pada bahasa pemrograman yang bersifat *weakly typed* atau mendukung konversi tipe implisit, nilai yang tersimpan dalam suatu variabel dapat berubah bentuk selama eksekusi berlangsung. Kondisi tersebut dapat menimbulkan ketidaksesuaian antara tipe yang diharapkan dan nilai yang sebenarnya terdapat di memori.

Untuk mengatasi permasalahan tersebut, interpreter dapat melakukan *Runtime Type Checking* – proses validasi tipe data selama eksekusi program berlangsung. Informasi tipe yang telah disimpan pada *Decorated AST* digunakan sebagai acuan untuk memastikan bahwa nilai aktual yang tersimpan di memori sesuai dengan tipe yang diharapkan.

BAB 2

Perancangan dan Implementasi

Pada Milestone 4, sistem Arion Compiler dikembangkan hingga tahap back-end, yaitu pembangkitan intermediate code dan eksekusi program menggunakan interpreter. Masukan dari tahap ini adalah Decorated AST dan tabel simbol yang dihasilkan oleh tahap analisis semantik. Keluaran yang dihasilkan berupa daftar instruksi intermediate code serta output aktual program.

Secara umum, alur kerja sistem adalah sebagai berikut:

- Source code diproses oleh lexer menjadi token.
- Token diproses oleh parser menjadi parse tree.
- Parse tree dianalisis oleh semantic analyzer menjadi Decorated AST dan symbol table.
- Decorated AST diterjemahkan oleh CodeGenerator menjadi intermediate code.
- Intermediate code dieksekusi oleh StackMachineInterpreter.

2.1 Intermediate Code Design and Generator

Setiap instruksi direpresentasikan oleh struktur Instruction yang memiliki empat komponen utama, yaitu nomor baris, opcode, level, dan argumen. Adapun opcode yang digunakan:

Opcode	Fungsi
INT	Menginisialisasi frame memori
LID	Memasukkan literal ke stack
LOD	Memuat nilai dari alamat memori
STO	Menyimpan nilai ke alamat memori
OPR	Menjalankan operasi aritmatika, logika, atau output
JMP	Melakukan lompatan tanpa syarat
JPC	Melakukan lompatan jika kondisi bernilai salah
CAL	Memanggil prosedur atau fungsi
RET	Mengakhiri frame eksekusi
LDA	Memuat alamat suatu variabel atau field ke stack, bukan nilainya
LDI	Memuat nilai secara tidak langsung dari alamat yang berada di stack

STI	Menyimpan nilai secara tidak langsung ke alamat yang berada di stack
CPY	Menyalin sejumlah slot memori, digunakan untuk assignment tipe terstruktur seperti array atau record
CHK	Memeriksa apakah indeks berada dalam batas valid array
REA	Membaca input dari pengguna untuk mendukung readln

Pembangkitan intermediate code diimplementasikan pada kelas *CodeGenerator* di *CodeGenerator.cpp*. Kelas ini menerima AST dan symbol table melalui fungsi *generate*, kemudian menghasilkan daftar *Instruction*.

```
(inside) ./src/Intermediate/CodeGenerator.hpp
```

```
#ifndef __CODE_GENERATOR_HPP__
#define __CODE_GENERATOR_HPP__

#include "Instruction.hpp"
#include "../Semantic/AST.hpp"
#include "../Semantic/SymbolTable.hpp"
#include <map>
#include <ostream>
#include <string>
#include <vector>

class CodeGenerator {
private:
    const SymbolTable *symbolTable;
    std::vector<Instruction> instructions;
    std::vector<std::string> errors;
    std::map<int, int> subprogramEntries;
    std::map<std::string, int> subprogramEntriesByName;
    std::map<int, int> subprogramParamCounts;
    std::map<std::string, int> subprogramParamCountsByName;
    std::map<int, int> subprogramParamSlots;
    std::map<std::string, int> subprogramParamSlotsByName;
    int currentFrameLevel;

    struct RuntimeLocation {
        int levelDiff;
        int address;

        RuntimeLocation(int l = -1, int a = -1) : levelDiff(l), address(a) {}
        bool isValid() const { return levelDiff >= 0 && address >= 0; }
    };

    int emit(const std::string &op, int level, const std::string &argument);
    int emit(const std::string &op, int level, int argument);
    void patchArgument(int instructionIndex, int targetLine);

    void generateNode(ASTNode *node);
```

```

void generateStatement(ASTNode *node);
void generateExpression(ASTNode *node);

void generateProgram(ProgramNode *node);
void generateBlock(BlockNode *node);
void generateDeclarationList(DeclarationListNode *node);
void generateAssignment(AssignNode *node);
void generateProcedureCall(ProcCallNode *node);
void generateIf(IfNode *node);
void generateWhile(WhileNode *node);
void generateRepeat(RepeatNode *node);
void generateFor(ForNode *node);
void generateCase(CaseNode *node);
void generateSubprogramDeclaration(SubprogramDeclNode *node);

int currentLine() const;
int globalFrameSize() const;
int frameSizeForBlock(const ASTNode *node) const;
RuntimeLocation runtimeLocation(const ASTNode *node) const;
RuntimeLocation runtimeLocationByName(const std::string &name) const;
bool generateAddress(ASTNode *node);
bool storeTopToTarget(ASTNode *node);
bool copyStructuredValue(ASTNode *source, ASTNode *target, int size);
bool generateLoadSlots(ASTNode *node, int size);
bool generateStoreInput(ASTNode *node);
bool isStructured(const ASTNode *node) const;
int storageSize(const ASTNode *node) const;
int operationForBinary(const std::string &op) const;
void generateSubprogramCall(const std::string &name, int tabRef, const
std::vector<ASTNode *> &args, bool leavesResult);
int subprogramEntry(int tabRef, const std::string &name) const;
int expectedParamCount(int tabRef, const std::string &name) const;
int expectedParamSlotCount(int tabRef, const std::string &name) const;
std::string formatCallArgument(int entry, int paramCount) const;
std::string formatReal(double value) const;
std::string quoteLiteral(const std::string &value) const;
std::string lower(const std::string &value) const;
void addError(const std::string &message);

public:
    CodeGenerator();

    const std::vector<Instruction> &generate(ASTNode *ast, const SymbolTable
&table);

    const std::vector<Instruction> &getInstructions() const { return
instructions; }
    const std::vector<std::string> &getErrors() const { return errors; }
    bool hasErrors() const { return !errors.empty(); }

    void print(std::ostream &out) const;
};

#endif

```



```
(inside) ./src/Intermediate/CodeGenerator.cpp
```

```
(source code terlalu panjang; bisa dibaca dari file)
```

Generator bekerja dengan menelusuri node AST dan menerjemahkan setiap jenis node menjadi instruksi yang sesuai. Program diawali dengan instruksi INT untuk alokasi memori global dan diakhiri dengan RET. Statement seperti assignment, procedure call, percabangan, dan perulangan diterjemahkan melalui fungsi khusus seperti generateAssignment, generateIf, generateWhile, generateRepeat, dan generateFor.

Untuk ekspresi, generator menggunakan pola evaluasi berbasis stack. Operand kiri dan kanan dimuat terlebih dahulu, kemudian operasi dilakukan menggunakan OPR. Sebagai contoh, ekspresi $a + b$ diterjemahkan menjadi LOD untuk a , LOD untuk b , lalu OPR 2 untuk operasi penjumlahan.

2.2 Perancangan Memori dan Addressing

Memori runtime dirancang menggunakan model stack frame. Setiap frame memiliki tiga slot kontrol utama:

Offset	Isi
0	Static link
1	Dynamic link
2	Return address

Slot setelahnya digunakan untuk parameter dan variabel lokal. Ukuran frame dihitung dari informasi pada symbol table, terutama psze untuk parameter dan vsze untuk variabel. Karena itu, setiap variabel pada Decorated AST harus memiliki referensi ke tabel simbol (tabRef) agar generator dapat menentukan alamat runtime-nya.

Alamat variabel dihitung dengan bentuk: $\text{alamat runtime} = 3 + \text{adr}$ pada symbol table. Untuk akses lintas scope, generator menghitung selisih lexical level sehingga interpreter dapat mengikuti static link menuju frame yang benar.

Struktur kontrol diterjemahkan menjadi kombinasi instruksi evaluasi kondisi, JPC, dan JMP. Pada if-else, generator membangkitkan kode untuk kondisi terlebih dahulu. Jika kondisi salah, JPC akan melompat ke blok else. Jika terdapat blok else, maka akhir blok then diberi JMP untuk melewati blok else.

Pada while, generator menyimpan posisi awal kondisi. Setelah badan perulangan selesai dieksekusi, JMP digunakan untuk kembali ke posisi awal tersebut. Jika kondisi bernilai salah, JPC akan melompat ke instruksi setelah badan perulangan.

Implementasi juga mendukung repeat, for, dan case. Pada repeat, badan perulangan dieksekusi terlebih dahulu sebelum kondisi diperiksa. Pada for, variabel loop diinisialisasi, dibandingkan dengan batas akhir, lalu diperbarui menggunakan operasi tambah atau kurang sesuai arah to atau downto.

2.3 Interpreter

```
(inside) ./src/Intermediate/Interpreter.hpp
```

```
#ifndef __INTERPRETER_HPP__
#define __INTERPRETER_HPP__

#include "Instruction.hpp"
#include <map>
#include <ostream>
#include <string>
#include <vector>

class StackMachineInterpreter {
private:
    struct StackValue {
        enum ValueType {
            INTEGER,
            REAL,
            STRING
        };

        ValueType type;
        int intValue;
        double realValue;
        std::string stringValue;

        StackValue();
        static StackValue fromInt(int value);
        static StackValue fromReal(double value);
        static StackValue fromString(const std::string &value);

        int asInt() const;
        double asReal() const;
        std::string toOutputString() const;
    };

    struct FrameInfo {
        int base;
        int limit;

        FrameInfo(int b = 0, int l = -1) : base(b), limit(l) {}
    };
};
```

```

std::vector<StackValue> stack;
std::vector<std::string> errors;
std::map<int, FrameInfo> frames;
int pc;
int sp;
int currentBase;
bool halted;
int maxSteps;
int callDepth;
int maxCallDepth;

int base(int level);
bool ensureIndex(int index);
bool validateProgram(const std::vector<Instruction> &instructions);
bool validateJumpTarget(int target, int instructionCount, const
std::string &op, int line);
bool validateDataAddress(int index, const std::string &action, int line);
bool validateDataBlock(int start, int size, const std::string &action, int
line);
bool pushIntChecked(long long value, const std::string &context);
bool push(const StackValue &value);
bool pop(StackValue &value);
bool popBinary(StackValue &left, StackValue &right);
int parseIntArgument(const Instruction &instruction);
void parseCallArgument(const Instruction &instruction, int &target, int
&paramCount);
StackValue parseLiteral(const std::string &argument);
StackValue parseInputValue(const std::string &token);
void executeOperation(int operation, std::ostream &out, int line);
void addError(const std::string &message);

public:
    StackMachineInterpreter(int stackSize = 2048, int stepLimit = 1000000, int
callDepthLimit = 1000);

    void reset();
    void execute(const std::vector<Instruction> &instructions, std::ostream
&out);

    bool hasErrors() const { return !errors.empty(); }
    const std::vector<std::string> &getErrors() const { return errors; }
};

#endif

```

(inside) ./src/Intermediate/Interpreter.cpp

(source code terlalu panjang; bisa dibaca dari file)

StackMachineInterpreter mengeksekusi intermediate code menggunakan pendekatan *stack-based virtual machine*, yaitu setiap instruksi dijalankan satu per satu berdasarkan nilai Program

Counter (pc). Ketika fungsi `execute()` dipanggil, interpreter terlebih dahulu melakukan inisialisasi melalui `reset()`, memvalidasi seluruh instruksi dengan `validateProgram()`, kemudian masuk ke siklus eksekusi (fetch-decode-execute cycle). Pada setiap iterasi, interpreter mengambil instruksi pada posisi pc, menaikkan nilai pc, lalu mengeksekusi instruksi sesuai opcode yang ditemukan. Selama proses ini, data dan hasil perhitungan disimpan pada struktur stack, sementara posisi elemen teratas stack dilacak oleh sp (Stack Pointer).

Instruksi yang berkaitan dengan manipulasi data bekerja langsung terhadap stack dan memori runtime. Instruksi LIT digunakan untuk mendorong (*push*) nilai literal ke stack, sedangkan LOD mengambil nilai variabel dari alamat tertentu dalam frame aktif dan meletakkannya ke stack. Sebaliknya, STO mengambil nilai dari puncak stack lalu menyimpannya ke alamat variabel yang dituju. Interpreter juga mendukung akses tidak langsung melalui LDA (load address), LDI (load indirect), dan STI (store indirect), sehingga memungkinkan implementasi pointer, array, atau record. Selain itu, instruksi CPY digunakan untuk menyalin blok data antar alamat stack, sementara CHK melakukan pemeriksaan batas untuk mencegah akses indeks di luar rentang yang diperbolehkan.

Untuk pengendalian alur program, interpreter menggunakan instruksi JMP dan JPC. Instruksi JMP memindahkan pc secara langsung ke alamat tujuan tanpa syarat, sedangkan JPC melakukan lompatan hanya jika nilai kondisi pada puncak stack bernilai nol (false). Mekanisme ini digunakan untuk mengimplementasikan struktur kontrol seperti if, while, dan repeat-until. Input dari pengguna diproses melalui instruksi REA yang membaca token dari stdin, mengonversinya ke tipe data yang sesuai, kemudian menyimpannya ke stack.

Pemanggilan prosedur dilakukan melalui instruksi CAL yang membentuk sebuah activation record atau frame baru pada stack. Frame ini menyimpan tiga informasi penting, yaitu Static Link (SL) untuk mengakses variabel pada lingkup leksikal induk, Dynamic Link (DL) untuk kembali ke frame pemanggil, dan Return Address (RA) yang menyimpan alamat instruksi setelah pemanggilan prosedur. Setelah frame terbentuk, `currentBase` dipindahkan ke frame baru dan pc diarahkan ke alamat prosedur yang dipanggil. Ketika instruksi RET dieksekusi, interpreter mengambil kembali nilai RA dan DL, mengembalikan pc ke lokasi pemanggil, memulihkan frame sebelumnya, serta menghapus frame yang telah selesai digunakan.

Operasi aritmetika, relasional, dan keluaran dieksekusi melalui instruksi OPR. Metode `executeOperation()` menangani berbagai operasi seperti negasi (NEG), penjumlahan, pengurangan, perkalian, pembagian, modulo, perbandingan (`=`, `!=`, `<`, `<=`, `>`, `>=`), serta operasi keluaran (WRITE dan WRITELN). Untuk setiap operasi biner, interpreter mengambil dua operand dari stack, melakukan perhitungan sesuai tipe datanya (integer, real, atau string), kemudian mendorong hasilnya kembali ke stack. Selama proses ini interpreter juga melakukan berbagai pemeriksaan keamanan seperti stack overflow, stack underflow, pembagian dengan nol,

overflow integer, validitas alamat memori, dan integritas frame. Jika terjadi kesalahan, fungsi `addError()` akan mencatat pesan kesalahan dan menghentikan eksekusi program.

2.4 Implementasi Bonus

Pembuatan deteksi dan penanganan terhadap *vulnerabilities* atau kerentanan mencakup enam jenis:

1. Pertama, stack overflow dicegah melalui batas ukuran stack dan batas kedalaman pemanggilan fungsi. Interpreter akan menghentikan eksekusi jika alokasi frame, operasi push, atau call depth melebihi batas yang ditentukan.
2. Kedua, stack underflow dicegah dengan validasi sebelum operasi pop. Jika interpreter mencoba mengambil nilai dari stack kosong, eksekusi dihentikan dengan runtime error.
3. Ketiga, stack smashing dan stack corruption dicegah melalui validasi alamat memori. Interpreter tidak mengizinkan operasi baca/tulis ke slot kontrol frame, yaitu static link, dynamic link, dan return address. Interpreter juga memeriksa keseimbangan stack saat RET; jika sp tidak sesuai dengan batas frame, program dianggap mengalami stack corruption.
4. Keempat, out-of-bounds variable access ditangani melalui instruksi CHK. Instruksi ini digunakan terutama pada akses array untuk memastikan indeks berada dalam batas lowBound sampai highBound. Jika indeks berada di luar batas, interpreter menghasilkan IndexOutOfBoundsException.
5. Kelima, invalid jump target dicegah melalui validasi program sebelum dan saat eksekusi. Target JMP, JPC, dan CAL harus berada dalam rentang instruksi yang valid.
6. Keenam, numerical overflow dan underflow ditangani pada literal integer, operand integer, input integer, serta operasi aritmatika integer seperti negasi, penjumlahan, pengurangan, perkalian, pembagian, dan modulus.

2.5 Alur Program

Berikut adalah kode `main.cpp` dari program:

```
./src/main.cpp

#include "Lexer/Lexer.hpp"
#include "Parser/Parser.hpp"
#include "Semantic/SemanticAnalyzer.hpp"
#include "Intermediate/CodeGenerator.hpp"
#include "Intermediate/Interpreter.hpp"
#include <string>
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
```

```

using namespace std;

int main(int argc, char *argv[]) {
    if (argc < 2) {
        cerr << "Usage: ./bin/arion-compiler [--parse|--semantic|--ic]
<input-file.txt>" << endl;
        cerr << "  Default: lexer only" << endl;
        cerr << "  --parse: run lexer + parser and output parse tree" << endl;
        cerr << "  --semantic: run parser + semantic analysis and output
decorated AST + symbol tables" << endl;
        cerr << "  --ic: run parser + semantic analysis + intermediate code
generation" << endl;
        cerr << "  Example: ./bin/arion-compiler test.txt" << endl;
        cerr << "  Example: ./bin/arion-compiler --parse test.txt" << endl;
        cerr << "  Example: ./bin/arion-compiler --semantic test.txt" << endl;
        cerr << "  Example: ./bin/arion-compiler --ic test.txt" << endl;
        return 1;
    }

    bool parseMode = false;
    bool semanticMode = false;
    bool intermediateMode = false;
    string filename;

    if (argc >= 3) {
        if (string(argv[1]) == "--parse") {
            parseMode = true;
            filename = argv[2];
        }
        else if (string(argv[1]) == "--semantic") {
            semanticMode = true;
            filename = argv[2];
        }
        else if (string(argv[1]) == "--ic") {
            intermediateMode = true;
            filename = argv[2];
        }
        else {
            filename = argv[1];
        }
    }
    else {
        filename = argv[1];
    }

    try {
        if (parseMode) {
            // Parser mode
            Lexer lexer(filename);
            Parser parser(lexer);
            ParseNode *tree = parser.parseProgram();

            // Print tree to stdout
            tree->print(cout);

            // Save tree to output file

```

```

        string outfile = filename;
        size_t lastDot = outfile.rfind('.');
        if (lastDot != string::npos) {
            outfile = outfile.substr(0, lastDot) + "_parse.txt";
        }
        else {
            outfile = outfile + "_parse.txt";
        }
        ofstream out(outfile);
        tree->print(out);
        out.close();
        cerr << "Parse tree saved to: " << outfile << endl;

        delete tree;
    }
    else if (semanticMode) {
        // Semantic Analysis mode
        Lexer lexer(filename);
        Parser parser(lexer);
        ParseNode *tree = parser.parseProgram();

        // Run semantic analysis
        SemanticAnalyzer analyzer;
        ASTNode *ast = analyzer.analyze(tree);

        // Print results to stdout
        analyzer.printResults(cout, ast);

        // Save results to output file
        string outfile = filename;
        size_t lastDot = outfile.rfind('.');
        if (lastDot != string::npos) {
            outfile = outfile.substr(0, lastDot) + "_semantic.txt";
        }
        else {
            outfile = outfile + "_semantic.txt";
        }
        ofstream out(outfile);
        analyzer.printResults(out, ast);
        out.close();
        cerr << "Semantic analysis saved to: " << outfile << endl;

        delete ast;
        delete tree;
    }
    else if (intermediateMode) {
        // Intermediate Code mode
        Lexer lexer(filename);
        Parser parser(lexer);
        ParseNode *tree = parser.parseProgram();

        SemanticAnalyzer analyzer;
        ASTNode *ast = analyzer.analyze(tree);

        if (analyzer.hasErrors()) {
            cerr << "Semantic analysis failed; intermediate code was not

```

```

generated." << endl;
    for (const auto &err : analyzer.getErrors()) {
        cerr << err << endl;
    }
    delete ast;
    delete tree;
    return 1;
}

CodeGenerator generator;
generator.generate(ast, analyzer.getSymbolTable());

generator.print(cout);
bool hasRuntimeErrors = false;
ostringstream runtimeOutput;
vector<string> runtimeErrors;
if (!generator.hasErrors()) {
    cout << endl << "=== Program Output ===" << endl;
    StackMachineInterpreter interpreter;
    interpreter.execute(generator.getInstructions(),
runtimeOutput);
    cout << runtimeOutput.str();
    if (interpreter.hasErrors()) {
        hasRuntimeErrors = true;
        cerr << "Runtime execution failed." << endl;
        for (const auto &err : interpreter.getErrors()) {
            runtimeErrors.push_back(err);
            cerr << err << endl;
        }
    }
}

string outfile = filename;
size_t lastDot = outfile.rfind('.');
if (lastDot != string::npos) {
    outfile = outfile.substr(0, lastDot) + "_ic.txt";
}
else {
    outfile = outfile + "_ic.txt";
}
ofstream out(outfile);
generator.print(out);
if (!generator.hasErrors()) {
    out << endl << "=== Program Output ===" << endl;
    out << runtimeOutput.str();
    if (hasRuntimeErrors) {
        out << endl << "=== Runtime Errors ===" << endl;
        for (const auto &err : runtimeErrors) {
            out << err << endl;
        }
    }
}
out.close();
cerr << "Intermediate code saved to: " << outfile << endl;

bool hasGenerationErrors = generator.hasErrors();

```



```

        delete ast;
        delete tree;

        if (hasGenerationErrors || hasRuntimeErrors) {
            return 1;
        }
    }
    else {
        // Lexer mode (default)
        Lexer lexer(filename);
        Token token = lexer.getNextToken();
        while (token.type != TokenType::END_OF_FILE) {
            string typeName = lexer.tokenTypeToString(token.type);

            if (token.type == TokenType::IDENT || token.type ==
TokenType::INTCON || token.type == TokenType::REALCON) {
                cout << typeName << " (" << token.value << ")" << endl;
            }
            else if (token.type == TokenType::STRING || token.type ==
TokenType::CHARCON) {
                cout << typeName << " ('" << token.value << "')" << endl;
            }
            else if (token.type == TokenType::UNKNOWN) {
                cout << typeName << " (" << token.value << ")" << endl;
            }
            else {
                cout << typeName << endl;
            }
            token = lexer.getNextToken();
        }
    }
}
catch (const exception &e) {
    cerr << "Error: " << e.what() << endl;
    return 1;
}
return 0;
}

```

Pengguna menjalankan compiler dengan mode intermediate code menggunakan perintah: `./bin/arion-compiler --ic <file-input>`.

Setelah file input diterima, program membuat objek Lexer untuk membaca source code dan mengubahnya menjadi token. Token-token tersebut kemudian diproses oleh Parser untuk membentuk parse tree berdasarkan grammar bahasa Arion. Parse tree ini belum cukup untuk dieksekusi karena hanya merepresentasikan struktur sintaksis program.

Tahap berikutnya adalah analisis semantik. Parse tree diberikan kepada SemanticAnalyzer untuk divalidasi secara semantik. Pada tahap ini dilakukan pemeriksaan seperti deklarasi identifier, kecocokan tipe data, validitas assignment, validitas ekspresi, scope variabel, serta pembentukan symbol table. Jika ditemukan kesalahan semantik, program menghentikan proses dan intermediate code tidak dibangkitkan.

Jika analisis semantik berhasil, sistem menghasilkan Decorated AST, yaitu AST yang telah dilengkapi informasi semantik seperti tipe data, level scope, referensi tabel simbol, ukuran storage, alamat variabel, batas array, dan informasi field record. Decorated AST inilah yang menjadi masukan utama bagi tahap intermediate code generation.

Selanjutnya, CodeGenerator menelusuri Decorated AST dan menerjemahkan setiap node menjadi instruksi intermediate code. Program diawali dengan instruksi INT untuk mengalokasikan frame memori, kemudian statement seperti assignment, percabangan, perulangan, pemanggilan prosedur, dan operasi output diterjemahkan menjadi instruksi seperti LIT, LOD, STO, OPR, JMP, JPC, CAL, dan RET. Untuk fitur tambahan seperti array dan record, generator juga dapat menghasilkan instruksi LDA, LDI, STI, CPY, dan CHK.

Setelah intermediate code selesai dibangkitkan, daftar instruksi dicetak ke layar dan disimpan ke file output. Jika tidak terdapat error pada proses pembangkitan kode, program melanjutkan ke tahap eksekusi menggunakan StackMachineInterpreter.

Interpreter menjalankan intermediate code dengan mekanisme stack machine. Setiap instruksi dibaca berdasarkan program counter, kemudian diinterpretasikan dan dieksekusi. Instruksi LIT memasukkan nilai ke stack, LOD mengambil nilai dari memori, STO menyimpan nilai ke memori, OPR menjalankan operasi aritmatika/logika/output, sedangkan JMP dan JPC mengatur perpindahan alur eksekusi. Jika program memanggil prosedur atau fungsi, interpreter membuat stack frame baru yang berisi static link, dynamic link, return address, parameter, dan variabel lokal.

Selama proses eksekusi, interpreter juga melakukan validasi runtime. Validasi ini mencakup pengecekan stack overflow, stack underflow, stack corruption, stack smashing, invalid jump target, akses array di luar batas, serta overflow atau underflow numerik. Jika terjadi runtime error, eksekusi dihentikan dan pesan error ditampilkan.

Apabila seluruh instruksi berhasil dijalankan hingga RET, output program ditampilkan dan juga dituliskan ke file hasil intermediate code.

BAB 3

Pengujian

input-1.txt	<pre>=== Intermediate Code === 0 INT 0 6 1 LIT 0 5 2 STO 0 3 3 LIT 0 10 4 STO 0 4 5 LOD 0 3 6 LOD 0 4 7 LIT 0 2 8 OPR 0 4 9 OPR 0 2 10 STO 0 5 11 LOD 0 5 12 OPR 0 14 13 RET 0 0 === Program Output === 25</pre>
input-2.txt	<pre>=== Intermediate Code === 0 INT 0 5 1 LIT 0 1 2 STO 0 3 3 LIT 0 0 4 STO 0 4 5 LOD 0 3 6 LIT 0 5 7 OPR 0 12 8 JPC 0 18 9 LOD 0 4 10 LOD 0 3 11 OPR 0 2 12 STO 0 4 13 LOD 0 3 14 LIT 0 1 15 OPR 0 2 16 STO 0 3 17 JMP 0 5 18 LOD 0 4 19 LIT 0 15 20 OPR 0 10 21 JPC 0 25 22 LIT 0 'ok' 23 OPR 0 14 24 JMP 0 27 25 LIT 0 'failed' 26 OPR 0 14 27 RET 0 0 === Program Output ===</pre>

	ok
input-3.txt	<pre> === Intermediate Code === 0 INT 0 8 1 LIT 0 1 2 STO 0 6 3 LIT 0 0 4 STO 0 7 5 LOD 0 6 6 LIT 0 3 7 OPR 0 12 8 JPC 0 34 9 LOD 0 6 10 LIT 0 10 11 OPR 0 4 12 LDA 0 3 13 LOD 0 6 14 CHK 1 3 15 LIT 0 1 16 OPR 0 3 17 OPR 0 2 18 STI 0 0 19 LOD 0 7 20 LDA 0 3 21 LOD 0 6 22 CHK 1 3 23 LIT 0 1 24 OPR 0 3 25 OPR 0 2 26 LDI 0 0 27 OPR 0 2 28 STO 0 7 29 LOD 0 6 30 LIT 0 1 31 OPR 0 2 32 STO 0 6 33 JMP 0 5 34 LOD 0 7 35 OPR 0 14 36 RET 0 0 === Program Output === 60 </pre>
input-4.txt	<pre> === Intermediate Code === 0 INT 0 4 1 JMP 0 10 2 INT 0 4 3 LIT 0 10 4 STO 0 3 5 LOD 1 3 6 LOD 0 3 7 OPR 0 2 8 STO 1 3 9 RET 0 0 10 LIT 0 1 </pre>

	<pre> 11 STO 0 3 12 CAL 0 2 13 LOD 0 3 14 OPR 0 14 15 RET 0 0 === Program Output === 11 </pre>
input-5.txt	<pre> === Intermediate Code === 0 INT 0 7 1 LIT 0 4 2 STO 0 6 3 LIT 0 99 4 LDA 0 3 5 LOD 0 6 6 CHK 1 3 7 LIT 0 1 8 OPR 0 3 9 OPR 0 2 10 STI 0 0 11 LDA 0 3 12 LIT 0 1 13 CHK 1 3 14 LIT 0 1 15 OPR 0 3 16 OPR 0 2 17 LDI 0 0 18 OPR 0 14 19 RET 0 0 === Program Output === === Runtime Errors === Runtime error: IndexOutOfBoundsException: index 4 is outside bounds 1..3 at line 6 </pre>
input-6.txt	<pre> === Intermediate Code === 0 INT 0 4 1 LIT 0 2147483647 2 LIT 0 1 3 OPR 0 2 4 STO 0 3 5 LOD 0 3 6 OPR 0 14 7 RET 0 0 === Program Output === === Runtime Errors === Runtime error: OverflowError: integer overflow during ADD at line 3 </pre>
input-7.txt	<pre> === Intermediate Code === 0 INT 0 5 1 LIT 0 0 2 STO 0 3 </pre>

	<pre> 3 LIT 0 0 4 STO 0 4 5 LOD 0 3 6 LIT 0 1 7 OPR 0 2 8 STO 0 3 9 LOD 0 4 10 LOD 0 3 11 OPR 0 2 12 STO 0 4 13 LOD 0 3 14 LIT 0 4 15 OPR 0 10 16 JPC 0 5 17 LOD 0 4 18 OPR 0 14 19 RET 0 0 === Program Output === 10 </pre>
input-8.txt	<pre> === Intermediate Code === 0 INT 0 5 1 LIT 0 0 2 STO 0 4 3 LIT 0 1 4 STO 0 3 5 LOD 0 3 6 LIT 0 4 7 OPR 0 12 8 JPC 0 18 9 LOD 0 4 10 LOD 0 3 11 OPR 0 2 12 STO 0 4 13 LOD 0 3 14 LIT 0 1 15 OPR 0 2 16 STO 0 3 17 JMP 0 5 18 LOD 0 4 19 OPR 0 14 20 RET 0 0 === Program Output === 10 </pre>
input-9.txt	<pre> === Intermediate Code === 0 INT 0 6 1 LIT 0 7 2 LDA 0 3 3 STI 0 0 4 LIT 0 8 5 LDA 0 3 6 LIT 0 1 7 OPR 0 2 </pre>

	<pre> 8 STI 0 0 9 LDA 0 3 10 LDI 0 0 11 LDA 0 3 12 LIT 0 1 13 OPR 0 2 14 LDI 0 0 15 OPR 0 2 16 STO 0 5 17 LOD 0 5 18 OPR 0 14 19 RET 0 0 === Program Output === 15 </pre>
input-10.txt	<pre> === Intermediate Code === 0 INT 0 5 1 LIT 0 10 2 STO 0 3 3 LIT 0 0 4 STO 0 4 5 LOD 0 3 6 LOD 0 4 7 OPR 0 5 8 OPR 0 14 9 RET 0 0 === Program Output === === Runtime Errors === Runtime error: Division by zero </pre>

BAB 4

Kesimpulan dan Saran

4.1 Kesimpulan

Pada Milestone 4 dan terakhir Tugas Besar IF2224 Teori Bahasa Formal dan Otomata, kami telah berhasil mengimplementasikan tahap intermediate code generation dan interpreter. Di tahap ini, Arion sudah dapat menerjemahkan input menjadi intermediate code dan dieksekusi untuk menghasilkan output aktual.

Intermediate Code Generator berhasil menerjemahkan Decorated AST menjadi instruksi linier berbasis stack machine. Instruksi yang dibangkitkan mencakup operasi dasar seperti INT, LIT, LOD, STO, OPR, JMP, JPC, CAL, dan RET, serta instruksi tambahan seperti LDA, LDI, STI, CPY, CHK, dan REA untuk mendukung akses tidak langsung, tipe data terstruktur, input, dan validasi runtime.

Interpreter yang diimplementasikan mampu menjalankan intermediate code dengan mekanisme stack machine. Interpreter dapat mengelola memori melalui stack frame, menjalankan operasi aritmatika dan logika, menangani percabangan dan perulangan, menjalankan procedure call, serta menghasilkan output melalui write dan writeln.

Selain fungsi utama, implementasi juga telah mencakup beberapa fitur bonus berupa deteksi kesalahan runtime. Interpreter mampu mendeteksi stack overflow, stack underflow, stack corruption, stack smashing, invalid jump target, akses array di luar batas, overflow integer, dan pembagian dengan nol.

4.2 Saran

Kami ingin berbagi beberapa saran kepada pembaca yang mungkin akan menghadapi tugas serupa di masa depan:

BillyO	rajin - rajin cek qna
Nathan	HORE TUGAS BESAR SUDAH SELESAI
Azri	Kapan ya sejago Sandwicheese
Rasyad	Kak william tolong ini nya di anuin

Lampiran

Repository: <https://github.com/AzriVz/CTF-Tubes-IF2224-2026>

Diagram Transisi DFA:

<https://drive.google.com/file/d/1Q8MlnukMm2jU2dEt5GNVJjzSQ2MCpIng/view?usp=sharing>

Tabel Pembagian Tugas:

Nama	NIM	Workload	Persentase
Billy Ontoseno Irawan	13524121	Laporan Bab 1-2	15%
Nathaniel Christian	13524122	CodeGenerator.hpp CodeGenerator.cpp Laporan Bab 2-4	35%
Azri Arzaq Pohan	13524139	CodeGenerator.hpp CodeGenerator.cpp Instruction.hpp Instruction.cpp Interpreter.hpp Interpreter.cpp	50%
Rasyad Satyatma	13524142		0%

Referensi

[CS 241 Lecture 9 - Maximal Munch and Context Free Grammars With thanks to Brad Lushman, Troy Vasiga and Kevin Lanctot](#)

[CS 241 Lecture 11 - Top Down Parsing, First and Follow With thanks to Brad Lushman, Troy Vasiga and Kevin Lanctot](#)